

Introducción a SQL

Gestión y Arquitectura de Datos





Structured Query Language

Lenguaje estándar para interactuar con bases de datos
relacionales

Características principales:



Lenguaje declarativo



Especificamos QUÉ queremos, no CÓMO



Estándar universal

Operaciones principales en SQL



SELECT

Consultar datos



INSERT

Insertar datos



UPDATE

Actualizar datos



DELETE

Eliminar datos



CREATE

Crear estructuras



ALTER

Modificar

Estructura básica de una consulta

```
1 SELECT columna1, columna2
2 FROM tabla
3 WHERE condicion;
```

Componentes principales:

 **SELECT**: especifica qué columnas queremos ver

 **FROM**: indica de qué tabla obtenemos los datos

 **WHERE**: filtra los resultados (opcional)

Ejemplo de consulta básica

```
1  -- Buscar empleados con salario mayor a 50000
2  SELECT nombre, apellido, salario
3  FROM empleados
4  WHERE salario > 50000;
```

nombre	apellido	salario
Juan	Pérez	55000

Operadores de comparación

=

Igual

<

Menor que

>

Mayor que

<=

Menor o igual

>=

Mayor o igual

<> o !=

Distinto

Operadores lógicos



AND

Ambas condiciones



OR

Al menos una



NOT

Negación

Ejemplo:

- `salario > 50000 AND nombre = 'Juan'`
- `ciudad = 'Buenos Aires' OR ciudad = 'Córdoba'`
- `NOT (edad < 18)`



Buscar valores en una lista

```
1  -- Buscar empleados con apellidos específicos
2  SELECT nombre, apellido
3  FROM empleados
4  WHERE apellido IN ('Garcia', 'Lopez');
```

nombre	apellido
Juan	García
María	López

Operador BETWEEN



Buscar valores entre dos rangos

```
1  -- Buscar empleados con salario entre 40000 y 50000
2  SELECT nombre, apellido, salario
3  FROM empleados
4  WHERE salario BETWEEN 40000 AND 50000;
```

nombre	apellido	salario
Marcos	Gómez	42000



Buscar patrones en texto

Comodines:

- % cualquier cantidad de caracteres
- _ un único caracter

```
1  -- Apellidos que terminan en G
2  SELECT nombre, apellido
3  FROM empleados
4  WHERE apellido LIKE '%G';
5
6  -- Apellidos que empiezan con G
7  SELECT nombre, apellido
8  FROM empleados
9  WHERE apellido LIKE 'G%';
```



Representa la ausencia de un valor

Operadores para NULL:

✓ **IS NULL**: verificar si es NULL

✗ **IS NOT NULL**: verificar si NO es NULL

```
1 -- Buscar empleados sin telefono
2 SELECT nombre, telefono
3 FROM empleados
4 WHERE telefono IS NULL;
```

nombre	telefono
Carlos	NULL

Ordenar resultados

Opciones de ordenamiento:

↑ **ASC**: ascendente (por defecto)

↓ **DESC**: descendente

```
1  -- Ordenar por edad descendente
2  SELECT nombre, apellido, edad
3  FROM empleados
4  ORDER BY edad DESC;
```

nombre	apellido	edad
Carlos	López	45
María	García	35
Juan	Pérez	30



Nombres temporales para tablas o columnas

Tipos de alias:

 **Alias de tablas:** empleados AS e

 **Alias de columnas:** nombre AS nombre_empleado

```
1  -- Usando alias
2  SELECT
3      e.nombre AS nombre_empleado ,
4      e.salario AS sueldo_mensual
5  FROM empleados e
6  WHERE e.salario > 50000;
```

Funciones de agregación



COUNT()

Contar registros



SUM()

Sumar valores



AVG()

Promedio



MAX()

Valor máximo



MIN()

Valor mínimo

Ejemplo de funciones de agregación

```
1  -- Estadísticas de empleados
2  SELECT
3      COUNT(*) as total_empleados,
4      SUM(salario) as total_salarios,
5      AVG(salario) as salario_promedio,
6      MAX(salario) as salario_maximo,
7      MIN(salario) as salario_minimo
8  FROM empleados;
```

total_empleados	total_salarios	salario_promedio	salario_maximo	salario_minimo
3	155000	51666.67	60000	45000



Agrupar resultados por columnas

```
1  -- Estadísticas por departamento
2  SELECT
3      departamento,
4      COUNT(*) as cantidad_empleados,
5      AVG(salario) as salario_promedio
6  FROM empleados
7  GROUP BY departamento;
```

departamento	cantidad_empleados	salario_promedio
Ventas	2	55000
IT	1	45000



Filtrar resultados de grupos

Diferencia con WHERE:

 **WHERE:** filtra registros individuales

 **HAVING:** filtra grupos después de GROUP BY

Ejemplo de HAVING

```
1  -- Departamentos con mas de 2 empleados
2  SELECT departamento, COUNT(*) as cantidad_empleados
3  FROM empleados
4  GROUP BY departamento
5  HAVING COUNT(*) > 2;
```

departamento	cantidad_empleados
Ventas	3
IT	4

Tipos de JOIN



Combinar filas de dos o más tablas



INNER JOIN

Solo coincidencias
entre ambas tablas



LEFT JOIN

Todos los de la
izquierda



RIGHT JOIN

Todos los de la
derecha



FULL JOIN

Todas las filas

Ejemplo de INNER JOIN

```
1  -- Combinar empleados con departamentos
2  SELECT e.nombre, d.nombre_departamento
3  FROM empleados e
4  INNER JOIN departamentos d ON e.id_departamento = d.id;
```

nombre	nombre_departamento
Juan	Ventas
María	IT
Carlos	Ventas

Ejemplo de LEFT JOIN

```
1 -- Todos los empleados, con departamento si existe
2 SELECT e.nombre, d.nombre_departamento
3 FROM empleados e
4 LEFT JOIN departamentos d ON e.id_departamento = d.id;
```

nombre	nombre_departamento
Juan	Ventas
María	IT
Carlos	Ventas
Pedro	NULL

Ejemplo de RIGHT JOIN

```
1 -- Todos los departamentos, con empleados si existen
2 SELECT e.nombre, d.nombre_departamento
3 FROM empleados e
4 RIGHT JOIN departamentos d ON e.id_departamento = d.id;
```

nombre	nombre_departamento
Juan	Ventas
María	IT
Carlos	Ventas
NULL	Marketing

Ejemplo de FULL JOIN

```
1  -- Todas las filas de ambas tablas
2  SELECT e.nombre, d.nombre_departamento
3  FROM empleados e
4  FULL JOIN departamentos d ON e.id_departamento = d.id;
```

nombre	nombre_departamento
Juan	Ventas
María	IT
Carlos	Ventas
Pedro	NULL
NULL	Marketing

¿Dudas?
¿Consultas?



Universidad de
SanAndrés